SmartFlow — Manuscript Revision Guide

Chapters 1 and 2 of "SMARTFLOW: A Simulation-Based Approach to Traffic Signal Optimization and Adaptive Routing Using Reinforcement Learning"

Prepared 26 October 2026 (Asia/Singapore) · Reference manuscript: `FINAL_SIGURO_CHAPTER1_AND_2.docx.pdf` (74 pages) · Page numbers are PDF positions counting the cover as page 1.

Basis: full reading of all 74 pages and all 22 embedded figures, the project history and migration record, the 8 October 2026 revision guide, and the SmartFlow source as of 26 October 2026. **Updated edition:** reflects every engine and RL fix now committed — deadlock and capacity fixes, maximum red, shared pedestrian rule, per-junction delay counter, engine version `native-5`, local reward `reward_v3`, training-equals-app action handling and the new training budget — and adds database, metric, security and testing facts checked against the code.

How to use this guide. Work through Section 1 first (it changes almost every page), then Sections 2–5 page by page. Label every statement you write as one of three kinds: **Implemented** (exists in the current software — present tense), **Evidence pending** (the software exists, but field data, calibration or the final comparison is not done), or **Proposed** (not built yet — future tense). Keep the six objectives; disclose what evidence is still missing instead of weakening them. The 8 October guide (`docs/reviews/MANUSCRIPT_REVISION_GUIDE_2026-10-08.md`) still holds the full 18-table data dictionary and detailed diagram specifications; this guide supersedes it where they disagree.

1. Changes that affect the whole manuscript

#	Change	Where and what to do
1	HIGH Replace SUMO / TraCI / PyTorch-DQN with the actual stack	The current system is a **browser application (React, TypeScript, Vite, Three.js)** connected to a **Python FastAPI backend** that runs SmartFlow's own **native microscopic traffic engine**, an **RL module (tabular Q-learning, DQL/DQN and PPO via Stable-Baselines3 on PyTorch)**, and **SQLite** records with model, recording and report files stored separately. SUMO currently appears in: scope (p7), literature (p11, 16, 18, 20–21), architecture (p32–35), conceptual framework (p35–36), requirements (p37–39), DFD prose (p44), WBS/Gantt (p29–31), technologies (p55–61), implementation and security (p63–67) and maintenance (p64, p70). Keep SUMO only as *literature* ("Kodama et al. used SUMO…").
2	HIGH Remove every emergency-vehicle item (owner decision, 24 Oct 2026)	Delete: the "Emergency Vehicle Priority" definition (p23); "emergency vehicles' status" as an input (p35–36, p40, p42, p44); "service of emergency vehicles" as an output (p36); functional requirement 1's emergency status (p37); use case 2.3 (Fig. 13); `emergency_condition` (Fig. 16, Table 3); emergency features and priority in data/RL text (p55–57, p60); "Emergency Vehicles" and "Emergency & Pedestrian Prioritization" in Fig. 11. Supported classes are now **car, motorcycle, tricycle, bus and truck**, plus pedestrians.
3	HIGH State the actual study area	Replace "a specific intersection and some connected junctions" with the real study area: **two signalized intersections on Osmeña Street — J1 Arellano×Osmeña and J2 Mabini×Osmeña** (provisional until confirmed by the adviser and field visit). Say the road geometry comes from OpenStreetMap and still needs field checking (lanes, turns, crossings, signal timing).

4	HIGH Make the RL and routing scope consistent	RL controls **one selected junction**; other junctions run their saved fixed-time plans. **Routing is a separate graph algorithm** (travel-time costs, closure avoidance, reroute threshold), not RL. Vehicles and pedestrians are **simulated traffic entities**, not AI agents. Consider a title that shows the split, e.g. *"SMARTFLOW: A Simulation-Based Decision-Support System for Reinforcement-Learning Traffic Signal Control and Adaptive Routing"* (needs adviser approval).
5	MED Separate "simulated real-time" from "live road data"	The dashboard updates live from the *simulation*; there are no live CCTV, GPS or sensor feeds and no control of real signals (p8 already says this — keep it). Field data enters only through the offline **Field Observer** tool (phone video → detector-suggested counts → human review → CSV import).
6	MED Tie every result to an engine version and network	Report results only from engine `native-5` (26 Oct 2026 onward) and name the network they ran on (provisional 2-junction, or the final confirmed one). Runs, recordings and models from older engines (`native-4` and earlier) are history: the app refuses to replay or load them, and their numbers must not be cited. Each trained model only works on the network and engine it was trained on.
7	MED Keep software evidence separate from research evidence	Passing tests prove the software behaves as specified; they do not prove the model matches Tagum traffic or that RL beats fixed-time. Use five separate kinds of evidence: (1) automated software tests, (2) browser workflow checks, (3) traffic-logic checks (no deadlock, counts balance), (4) field calibration and validation, (5) controller comparison. Only (1)–(3) exist today.

2. Chapter 1 — page-by-page changes

Page	Current text / issue	Change
Cover	"June 2026"	Use the actual submission month.
p3	"Validation of the research area will be conducted by field observation and consultation…"	Keep, but name the two study intersections and the Field Observer procedure (single stationary phone recording per take, reviewed counts).
p4	Citations [19] (DNSC RDE Agenda), [3] (DOST HNRDA), [16] (CHED) point to unrelated papers	Add the correct institutional sources and renumber. [19] is an IoT traffic-light paper; [3] is a predictive-analytics paper.
p5	General objective sentence is broken: "To create SMARTFLOW, The main objective of this project is…"	Rewrite as one sentence, e.g. "To develop and evaluate SMARTFLOW, a simulation-based decision-support system for a bounded Tagum City study area that uses observed traffic data to test reinforcement-learning signal control and adaptive routing under ordinary demand, events and road disruptions." Keep the six specific objectives; add that inputs are field-observed and that controllers are compared against a fixed-time baseline.
p6–7	Significance	Present benefits as *intended* decision support; LGU / traffic office is the primary user (matches the new dashboard). No real-world benefit is established yet.
p7–9	Scope: "developed using SUMO"; vehicles/pedestrians generated from acquired data	Rewrite the technology paragraph (see Appendix A.1) and add the model description and limits (see Section 6). Mention the offline Field Observer and that SmartFlow does not control physical signals.
p11	"In SMARTFLOW, SUMO will be used to simulate the selected intersection…"	Change to the native engine; keep the SUMO literature paragraph as background.

p14–16	"SMARTFLOW… intersection in Tagum City which has already been verified to be similar to real-life circumstance" (p16)	Unsupported. Write "will be validated against field observations of the selected intersections."
p15	Fig. 2 text describes standard DQN, Multistep DQN and "unbiased sample variance of 0.02"	The image shows only one "Proposed method" curve (trials 500–1000). Use the original figure with all curves or rewrite the text to match the image.
p16–17	Fig. 3 text: "CNN encoder, interaction graph (GNN), MLP trajectory decoder"; Fig. 4 text: red/blue predictions vs green ground truth	Fig. 3 image is a three-panel overview strip (layout → TrafficSim → rendering); Fig. 4 shows manoeuvre examples. Replace the images with the matching TrafficSim figures or rewrite the captions and text.
p19	Fig. 6: "real-time congestion monitoring application… smartphone interface (low/medium/high)"	Image shows a table-top prototype and LCD board, not a phone app. Fix caption or image.
p14–19	All literature figures	Add "Source: Author et al. [n]" under every borrowed figure (Figs. 1–6).
p18, 20	"SMARTFLOW is based only on the simulation system using SUMO…" / "in case of emergencies"	Change to the native engine; remove the emergency example.
p21–22	Table 1: SMARTFLOW "RL + Agent-Base", "SUMO"; "Event-Baseed"; "Traffic Signal Placements Evaluation ✓"	Main technology: "RL + native microscopic simulation"; environment: "Native Python engine (OSM-derived network)". Fix the typo. Qualify placement evaluation as "comparison of candidate signal configurations" — there is no automatic placement optimizer.
p23–24	Definitions	**Remove:** Emergency Vehicle Priority; Autonomous Agent and Heterogeneous Agent Behaviour (or mark as literature-only); SUMO (keep only if the literature needs it). **Add:** native traffic engine, approach, phase / service, fixed-time baseline, calibration, validation, throughput (completed trips; trips per hour), maximum red, on-call pedestrian phase, observation provenance, simulated real-time data.
Lit. review	No sources for the methods SmartFlow actually uses	Add: Intelligent Driver Model (Treiber, Hennecke & Helbing, 2000); Q-learning (Watkins & Dayan, 1992); DQN (Mnih et al., 2015); PPO (Schulman et al., 2017); Stable-Baselines3 (Raffin et al., 2021); Webster's fixed-time signal timing (1958); FHWA microsimulation calibration guidance; OpenStreetMap.

3. Chapter 2 — text changes

Page	Issue	Change
p25–30	Methodology phases describe SUMO configuration, vehicle/pedestrian *agents*, Python–TraCI integration	Keep the hybrid Waterfall/Agile method if it is accurate. Replace implementation tasks with: native engine, React/API, Field Observer and data import, calibration/validation, RL training, controlled experiments. Add the feedback loop "field observation → baseline validation → model refinement".
p30	"both fixed-time and **SMART-based** signal controls"	"fixed-time and RL-based signal control".
p31	"begins with the **four** Waterfall phases (i.e., Planning, Analysis, Design)"	Three phases.
p32–34	Architecture: SUMO simulation layer; Python + TraCI control layer; PyTorch intelligence layer; SQLite; outputs; "prioritize emergency and pedestrian movement"	Rewrite using Section 6 and Appendix A.2. Live state is held in the engine's memory; SQLite stores records and summaries; model and recording files are kept on disk.
p35–37	Conceptual framework inputs: traffic density, pedestrian density, emergency vehicle status, constraint type/severity/lane	Inputs: OSM-derived road network, reviewed field observations (CSV), vehicle mix (5 classes), demand windows, signal plans (green, min/max green, yellow, all-red, offset, maximum red, pedestrian rule), timed lane events (closure, reopening, speed restriction), controller choice, seed. Outputs: KPIs vs fixed-time, per-junction queues/phases, reports and CSV/JSON exports.
p37–38	Functional requirements 1 (emergency), 3 (SUMO), 4 (agent interactions), 6–8 ("continuously learning while running")	Use observable requirements: execute a saved scenario in the native engine; preserve complete settings; offline training, saved-model inference, separate evaluation; the policy only *requests* a service and the engine enforces safety timing; record pending/unfinished demand; replay recordings; role-based permissions; backup/restore.
p38–39	NFR 5: "operating systems like SUMO, Python, TraCI, PyTorch"; NFRs without targets	Separate host OS from languages/frameworks. Give each NFR a measurable check (e.g. simulation speed in decisions per second, page load, backup restore succeeds) — only targets you actually measure.
p40	"under **MAP**", "Reverse Engineers", "RE"	Garbled — replace with "scenario execution" and "RL controller".
p44	DFD prose: SUMO-based simulation; emergency vehicle state	Native engine; remove emergency; add training jobs, recordings, observations and calibration records.
p45	HIGH Three paragraphs about an **airport FAA ASDE-3 "Deficiency Processing System"**	Pasted from an unrelated document. Delete and replace with the Signal Control Processing description (Appendix A.3).
p46–47	ERD text: "does not have role-based access control at this time"; "dependent signal control"; one-to-one metric/report claims	The system *has* role-based access control (roles, permissions, sessions, audit log). Use "fixed-time control". Remove one-to-one claims: one run can have several `run_metrics` rows (no unique constraint), and there is no separate signal-mode, RL-output or performance-report table. See Section 6.6.

p49–55	Data dictionary Tables 2–9 (“accuont_id”; Table 9 `generated_by` not in ERD; “User” vs “USERS_ACCOUNTS”)	Replace with the real schema: **18 tables, 163 columns, 15 declared foreign keys** (users, roles, permissions, user_sessions, login_attempts, audit_logs, backups, notifications, system_settings, scenarios with `engine_config` and `network_id`, scenario_constraints, simulation_runs, run_metrics, rl_models, rl_checkpoints, rl_training_jobs, rl_training_job_items, calibration_records). Write SQL types as they are (TEXT, INTEGER, REAL — no VARCHAR lengths, JSON stored as TEXT, timestamps stored as TEXT). Do **not** describe the leftover columns `scenarios.emergency_mode`, `run_metrics.emergency_clearance_time` and `run_metrics.signal_phase_efficiency` as working features: they come from the retired design and are never filled in. Key facts are in Section 6.6; column-by-column detail is in the 8 Oct guide, Section 10 (its emergency and observation descriptions are outdated).
p55–57	Data collection “takes place in SUMO”; features include emergency vehicles and “road fall-back status”	Describe **two data streams**: (1) field observations — Field Observer → reviewed CSV → scenario demand; (2) simulation outputs — engine metrics for evaluation and RL observations. Describe the RL interface from Section 6.
p56–57	Reward and learning described generally; “priority for emergency vehicles”	State the real observation (22 values), actions (5 services) and engine safety rules (Section 6.2). **Write the reward equation now** — the local, per-interval reward `reward_v3` is implemented (Appendix A.4). State the training setup: about 101,000 decisions per run (560 episodes × 900 s after a 300 s warmup, 5 s decisions), training seeds 11–55, held-out evaluation seeds 101–105 on a different saved scenario (Appendix A.5). Remove emergency priority.
p57–61	Technologies: SUMO, RL, Python & TraCI, PyTorch (“DQN model to be implemented”), SQLite	Technologies: React/TypeScript/Vite, Three.js, FastAPI, native Python engine, Gymnasium environment, Stable-Baselines3 (DQN, PPO) on PyTorch, tabular Q-learning, SQLite, OpenStreetMap, Field Observer (detector-assisted review).
p62–64	Testing/implementation: SUMO platform, TraCI, “desktop local environment”	Report the actual automated suites as *software* evidence, with date and scope (Section 6.8, Appendix A.7): on 26 Oct 2026, **178 backend tests** (with 407 sub-cases) and **105 frontend tests** passed. Use a test table: requirement → procedure → input → expected behaviour → dated result → limitation. Do not reuse old counts (“40 passed”, “107 passed”). Deployment: local web application in a browser, one API worker.
p65	“System Security Plan provided in **Table IV**”	“Table 10”.
p65–68	Security controls “will” be implemented; A.13 “SUMO, Python, TraCI”	Describe what exists, in present tense (Section 6.7): scrypt password hashing, password-strength rules, login lockout after repeated failures, one-hour HTTP-only session cookies, account status and forced password change, role-based page/action permissions, audit logs, backup/restore bundles. Hashing is one-way — never call it “encryption”. Call ISO/IEC 27001 and 25010 “aligned practices”, not certification. Remove SUMO/TraCI.
p68–70	Maintenance: “compatibility with SUMO”; “simulation calibration” as routine	Compatibility with Python, Node and browsers; calibration is a research step with recorded evidence (Calibration & Validation page), not periodic tweaking.

New	No description of the user interface	Add a UI section with screenshots: LGU-oriented dashboard (KPI cards vs fixed-time, minimal map with phase/timer chips, junction tags, network map, junction card with Signal phases / Queue info / Performance tabs, Signal control card, setup and replay), Scenarios editor, RL Training, Runs & Reports, Compare Runs, Calibration & Validation, admin pages.
New	No evaluation protocol	Fixed-time baseline from **observed Tagum signal timing**; **same pedestrian rule** for both controllers; matched seeds; network-wide *and* J1-only results (J1 delay = stopped vehicle-seconds per passing vehicle, from the per-junction counter); report pending/unfinished demand and the two watchdog metrics (stuck vehicles, gridlock — both must stay 0); engine `native-5`; warmup, measurement window and seeds stated. Draft text: Appendix A.5.
New	No description of the traffic model itself	Add a short “Simulation model” subsection (from Section 6.1): car-following, junction rules, signal timing and its defaults, routing, and the limits. Cite the Intelligent Driver Model. Add the synthetic plausibility check (discharge headways) as model verification, clearly not as calibration.

4. Figures — what is wrong and what to draw instead

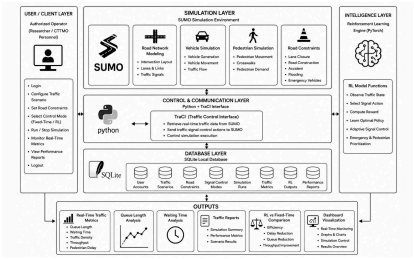


Fig. 11 — SUMO, TraCI, PyTorch, Flooding, Emergency Vehicles.

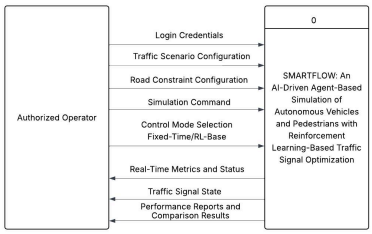


Fig. 14 — process box still shows the *original* title.

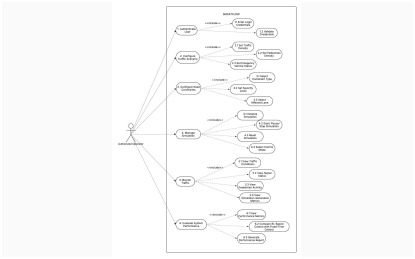


Fig. 13 — "Set Emegency Vehicle Status"; one actor only.



Fig. 19 "Python and TraCI" — shows the old `SumoSimulationEngine` with `emergency_mode` / `flooding`.

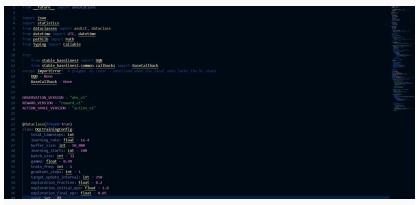


Fig. 18 "RL" — old training code with `obs_v1`.

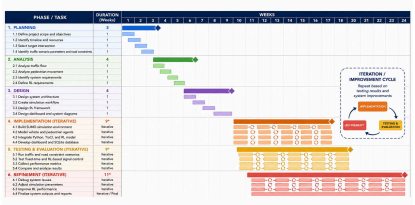


Fig. 10 Gantt — week 19 missing; SUMO/TraCI tasks.

Figure	Problem	Change
7 Hybrid method	Typo "optomize"	Keep the structure; add "field observation → baseline validation → refine".
8 Team	Photos labelled with roles only	Add each member's name under the photo; assign field observation, review and evaluation responsibilities.
9 WBS	"4.1 Build Simulation in SUMO", "4.3 Integrate Python, TraCI and RL Model", "Vehicle and Pedestrian Agents"; typos "1.3 Selec target", "SimulationWorkflow"	Work packages: site agreement; field pilot and coverage review; native engine and baseline model; React/API/persistence; observation import; calibration and validation; RL training; controlled experiments; findings and manuscript; acceptance and backup rehearsal.
10 Gantt	Header jumps from week 18 to 20 (**week 19 missing**); SUMO/TraCI tasks; 24-week plan	Redraw with real dates and status of the actual tasks.
11 Architecture	SUMO, TraCI, PyTorch only, Flooding, Emergency Vehicles, Emergency & Pedestrian Prioritization	Browser (React + Three.js) → FastAPI (auth, scenarios, runtime, WebSocket) → native engine + RL module (QL/DQL/PPO, Stable-Baselines3) → SQLite + model/recording/report files. Side path: Field Observer → reviewed CSV → scenario.
12 Conceptual	"SUMO Traffic Simulation" in Process	Input: OSM network, field observations, signal plans, events, controller. Process: native simulation, fixed-time vs RL, rerouting, calibration/validation. Output: KPIs vs baseline, per-junction results, comparison, reports.
13 Use case	Emergency use case; "Set Severity Level"; only "Authorized Operator"	Actors: Operator / researcher and Administrator. Use cases: authenticate; manage scenarios; run, pause, stop, replay; train RL model; compare runs; record calibration/validation; export reports; manage users and roles; view audit log; backup/restore.

14 Context	Process title is the **old** title (“AI-Driven Agent-Based Simulation of Autonomous Vehicles and Pedestrians…”); “RL-Base”	Use the current title; flows: credentials, scenario settings, observations (CSV), commands, controller choice → metrics, signal states, reports, exports.
15 DFD	“Comaprison Resaults”; SQLite as a hop for every live step	Add training jobs, recordings, observations and calibration stores; live state stays in engine memory.
16 ERD	Eight entities that do not match the database; `emergency_condition`	Draw the real 18 tables as three figures: **16a accounts and administration** — roles 1–N users, roles 1–N permissions, users 1–N user_sessions, users 1–N notifications, users 1–N backups (creator); audit_logs, login_attempts and system_settings stand alone (no declared FK). **16b scenarios, runs and calibration** — users 1–N scenarios (creator), scenarios 1–N scenario_constraints, scenarios 1–N simulation_runs, users 1–N simulation_runs, simulation_runs 1–N run_metrics, users 1–N calibration_records (baseline run and site are inside its JSON, not FKs). **16c learning records** — rl_models 1–N rl_checkpoints, rl_training_jobs 1–N rl_training_job_items, job items → rl_models (optional), simulation_runs → rl_models (optional). Mark run user and scenario links as optional (both columns allow NULL). Add a note that road networks, models, recordings and imported CSVs are *files* referenced by these tables.
17–21 Technologies	Screenshots of the **old system**: SUMO netedit (17), old training code (18), `SumoSimulationEngine` captioned “Python and TraCI” (19), runtime helper captioned “PyTorch” (20), old `database.py` with flooding config (21)	Replace with current screenshots (dashboard, 2D/3D map, scenario editor, RL Training, Field Observer) and two simple diagrams: the RL loop (observe 22 values → policy requests one of 5 services → engine safety rules decide → 5 s later the J1 reward from discharged vehicles, stopped time, longest queue, waiting pedestrians and switching) and the engine step (demand → car-following → junction occupancy → signals → metrics). Describe DQL as Stable-Baselines3 DQN and PPO as Stable-Baselines3 PPO, both on PyTorch; Q-learning is a table, not a neural network.
All	“Fig. 1” style in Chapter 1, “Figure 7” style in Chapter 2	Use one style and update the list of figures and tables.

5. References and editorial

Issue	Action
[6], [20] and [32] are the same Kodama et al. (2022) paper	Keep one entry and renumber all citations.
[7] and [21] share arXiv:2603.12096 but list different authors and titles	Check the real paper; keep one correct entry.
[19], [3], [16] cited for DNSC RDE, HNRDA and CHED agendas but point to unrelated papers	Add the actual institutional sources.
[22] Miller, [23] Al-Wajih, [24] Williams (e.g. "IEEE Transactions on Smart Cities")	**Verify each one exists** (DOI or URL). Remove any that cannot be found.
[4], [8], [9] appear never to be cited; [15], [16] lack a year	Cite or remove; complete the entries.
Literature performance figures	Attribute them to their studies; they are not SmartFlow results.
Typos	Fig. 7 "optomize"; Fig. 9 "Selec", "SimulationWorkflow"; Fig. 13 "Emegency"; Fig. 14 "RL-Base"; Fig. 15 "Comaprison Resaults"; Table 1 "Event-Baseed"; Table 2 "accuont_id"; Fig. 19 caption "TraCI".
Tense and terms	Present tense for implemented behaviour, future tense for proposed work, past tense only for completed procedures with dates. Use consistent terms: native engine, selected junction, service request, simulation-generated data, field observation, calibration, validation.

6. Current system facts to use when rewriting

6.1 Traffic engine (engine version `native-5`, 26 October 2026)

- Deterministic microscopic simulation in metres and seconds, 0.1 s steps; the same seed gives the same run; road network derived from OpenStreetMap; **one modelled lane per permitted direction**.
- Vehicle classes: car, motorcycle, tricycle, bus, truck (different lengths and speed factors); pedestrians with crossings and waiting.
- Car-following: Intelligent Driver Model (IDM) style; vehicles stop only behind a leader or at a stop line.
- Junctions: only vehicles from the approach that has green may be inside together; other approaches and pedestrians wait until it is empty; a vehicle enters only if everything on its exit road plus itself fits (no "blocking the box"). Through movements keep road speed; turns slow to a turning speed (calibration parameter).
- Signals: one approach served at a time plus a pedestrian phase; green → yellow → all-red; minimum and maximum green; a demanded approach is given green after at most the **maximum red** (default 120 s, or the plan's own fixed-time red if longer); pedestrian phase is **on-call** by default (skipped when nobody waits) and the same rule applies to fixed-time and RL. Option: `every_cycle` serves the walk phase every cycle.
- Default fixed-time plan (a *study assumption*, to be replaced by the observed Tagum timing): 14 s green, 3 s yellow, 1 s all-red per approach; minimum green 10 s, maximum green 60 s; pedestrian phase 14 s plus 1 s all-red; a full cycle of four approaches plus pedestrians is 87 s.
- Built-in fault checks (always reported): *vehicles stuck inside a junction* (stopped there for over 30 s) and *gridlock duration* (no movement for over 60 s while a vehicle waits on green for space). Both should read 0; a non-zero

value means an engine fault or a true jam, and the run must be investigated, not reported.

- Per-junction delay counter: stopped vehicle-seconds at each junction and approach, used for the RL reward and for J1-only results.
- Routing: graph shortest path by travel-time costs; closures avoided; congestion rerouting with improvement threshold and cooldown.
- Synthetic plausibility check (not calibration): a saturated through lane discharges about one car per 2.4 s (~1,480 veh/h/lane); turning ~2.9 s; motorcycles ~2.0 s; buses ~3.6 s.
- Limits to disclose: no lane changing, overtaking or motorcycle filtering; no permissive (yielding) turns; parameters not yet calibrated to Tagum data.

6.2 Reinforcement learning

- Algorithms: tabular Q-learning, DQL/DQN and PPO (Stable-Baselines3 on PyTorch), trained offline in a Gymnasium environment. Live dashboard runs only *use* a saved model; they never update its weights.
- Observation (contract `native-local-22-v2`): 22 values — current service (5), elapsed and remaining time (2), queue per approach (4), average wait per approach (4), active vehicles per approach (4), waiting pedestrians (1), network identity (2). Q-learning uses a 15-part table state built from the same information. Note for the manuscript: the per-approach wait is each vehicle's whole-trip stopped time.
- Actions (`protected-service-5-v1`): 5 service *requests* — north, east, south, west, pedestrians. The policy never sets lamps directly. The engine decides: during minimum green the request waits its turn, during yellow/all-red it is ignored, a request for a missing approach is refused, and the maximum-red rule can override it.
- Same behaviour in training and in the app: DQL and PPO apply their own chosen action in both (the engine's rules keep it safe); Q-learning picks only from currently allowed actions in both. Each decision records what the policy asked for and what the engine did, so overrides can be counted.
- Reward (`reward_v3`): computed for the controlled junction only, over each 5 s decision interval — equation in Appendix A.4. Synthetic check (default network, medium traffic, 2 seeds): a sensible round-robin policy scored −0.008, always-north −0.172 and always-pedestrian −0.185, matching J1 delay of 50 vs 132 vs 140 stopped seconds per passing vehicle. Label this as reward verification, not a controller result.
- Training defaults: 560 episodes × 900 s after a 300 s warmup with 5 s decisions ≈ 101,000 decisions per run; training seeds 11, 22, 33, 44, 55; evaluation seeds 101–105 on a different saved scenario (the app refuses overlapping seeds or the same scenario). Measured cost: about 5–8 decisions per second with learning, so roughly 3.5–5.5 hours per algorithm.
- Scope: one selected junction; each model is tied to the exact network and engine version it was trained on.
- *Evidence pending:* retraining QL, DQL and PPO on the confirmed study network, and the final comparison against fixed-time. No trained model is currently valid for `native-5`.

6.3 Metrics — use these exact definitions

Dashboard / report name	Unit	Definition (from the metric catalogue)
Average wait	s	Stopped time (speed ≤ 0.1 m/s) since the measurement started, divided by completed plus active vehicles. Vehicles still waiting at the network edge are excluded and reported separately.
Average queue	vehicles	Mean number of stopped vehicles per incoming lane, averaged over every step. Not a queue length in metres and not a network total.
Maximum queue	vehicles	Largest stopped count on any single lane at any step.
Completed trips / throughput	vehicles	Trips that left the network during the measurement window. Not a stop-line count.

Trips per hour	veh/h	Completed trips × 3600 ÷ measured seconds — a scaled rate, not an observed hourly count.
Average travel time	s	Entry-to-exit time of trips completed in the window; unfinished trips excluded.
Pedestrian delay	s	Waiting time before crossing, per completed plus active pedestrian; crossing time excluded.
Pending / dropped / unfinished demand	vehicles	Vehicles waiting to enter, requests dropped at the edge, and vehicles still in the network at the end. Always report these next to averages — a controller can look good by leaving vehicles unfinished.
J1 delay (evaluation)	s/vehicle	Stopped vehicle-seconds at J1 ÷ vehicles that passed J1.
Stuck vehicles, gridlock, conservation error	vehicles, s	Fault checks; must be 0. Zero proves consistent accounting, not realism.

Every KPI shown as "— / unavailable" has no valid population yet; never report it as zero. The dashboard compares a run with a fixed-time run of the same scenario and seed, on the same engine version.

6.4 Data, evidence and users

- Field Observer (offline): one stationary phone video per take; detector suggests people, cars, motorcycles, buses and trucks (no tricycle or jeepney class — reviewer corrects); counting lines; human review; provisional export to CSV.
- Calibration & Validation page: stores baseline-versus-field comparisons and independent-period checks with frozen evidence; a numerical fit is not proof of field accuracy.
- CSV import: aggregate counts are spread evenly within each time bin and entered as cars; explicit trip files keep each vehicle's class and entry time. Missing or occluded periods stay unknown — never zero. One phone covers one view per take, so sequential takes are not simultaneous network counts.
- Users: LGU / traffic office operator and administrator. The system has two built-in roles, `admin` and `user`; "researcher", "planner" and "CTTMO personnel" describe the audience, not separate roles.

6.5 User interface (for the new UI section and screenshots)

- Simulation dashboard for LGU staff: four KPI cards (average wait, average queue, trips per hour, average travel time) with the % change against fixed-time; a clean map (2D by default, 3D optional) with three chips — intersection, signal phase, phase timer — and J-tags; a control bar (start, pause, stop, speed, clock).
- Right column: network map; selected-junction card with photo and three tabs — *Signal phases*, *Queue info*, *Performance*; Signal control card (fixed-time, or QL/DQL/PPO when a compatible model exists).
- Other pages: Scenarios (editor), Compare Runs, Runs & Reports, RL Training, Calibration & Validation, and admin pages (users, roles, audit log, backup/restore, settings).

6.6 Database and file storage

- SQLite, 18 tables, 163 columns, 15 declared foreign keys (counted on a freshly created database, 26 Oct 2026). Unique: `roles.name`, `users.username`, `users.email`. Not unique: `run_metrics.run_id`, `user_sessions.token`.
- Native scenario settings are one validated JSON document (`scenarios.engine_config`): network and controlled junction, signal plans, lane events, vehicle mix, trips and demand windows, routing settings, pedestrian settings, and the data source (synthetic or observed, file hash, collection period).
- Kept as files, referenced by the tables: road network JSON (`data/networks/`), trained models and checkpoints with their metadata, recordings (compressed JSONL plus a manifest), imported CSVs (named by hash), training logs and evaluation summaries, backup bundles.
- Live traffic state lives in the engine's memory, not in SQLite; the database stores run records and summary metrics at the end.

- Leftover columns from the retired design (`scenarios.emergency_mode`, `run_metrics.emergency_clearance_time`, `run_metrics.signal_phase_efficiency`) are never filled — omit them from results or mark them as unused.

6.7 Security as implemented

- Passwords stored as scrypt hashes (one-way); minimum 8 characters with upper-case, lower-case and digit rules.
- Login lockout after 5 failed attempts per username and IP, starting at 5 minutes and growing up to 1 hour; attempts kept 30 days.
- Sessions: random 256-bit token in an HTTP-only cookie, 1-hour timeout renewed on activity; accounts can be deactivated, and an administrator can require a password change at next login.
- Role-based page and action permissions checked by the backend; audit log of user and system actions; backup and restore of the database together with its referenced files.
- Not claimed: encrypted storage at rest, certification, or HTTPS unless a deployment actually uses it.

6.8 Software verification (26 October 2026)

- Backend: 178 automated tests (407 sub-cases) passed — engine rules, deadlock and capacity checks, maximum red, pedestrian rule, reward, training settings, API, permissions, backup/restore.
- Frontend: 105 automated tests passed, plus TypeScript and lint checks; browser checks of the dashboard on the provisional 2-junction network.
- Pipeline trial: short QL, DQL and PPO training runs saved, reloaded and evaluated against fixed-time end to end. These are software checks, not research results.

7. Objective status — what you can and cannot claim yet

Objective (p5)	Implemented in software	Evidence still pending
1. Model a real Tagum intersection	OSM-derived network; provisional 2-junction study area (J1, J2); 2D/3D views	Final site confirmation; field-checked lanes, turns, crossings and signal timing
2. Use actual traffic data	Field Observer; CSV import with provenance; demand from observations	Reviewed recordings with coverage; conversion to demand; calibration and independent validation
3. Configurable interface	Scenario editor, signal plans, events, controller choice, seed; LGU dashboard	Usability check with intended users
4. Closures and rerouting	Timed closures/reopenings, speed restrictions, adaptive rerouting	Site-specific disruption cases and comparisons
5. Event scenarios	Demand windows (ordinary/peak/recovery) and scheduled events	Observed or documented event demand
6. RL signal learning	QL/DQL/PPO training, inference and comparison tools; local reward; training budget; same action handling in training and app; engine `native-5`	Retraining on the confirmed study network, fixed-time baseline from observed timing, final matched-seed comparison (network-wide and J1)

> **Do not claim yet:** calibration against Tagum data, the final study sites as confirmed, RL outperforming fixed-time, real-world benefits, or a signal-placement recommendation. Write these as planned work or "evidence pending".

8. Recommended order of work

1. Section 1 cross-cutting changes (stack, emergency removal, study area, RL/routing scope).
2. Delete the p45 airport text and fix p40, p30, p31, p65 wording; fix typos and figure captions.
3. Redraw Figures 9–16 and replace 17–21; add UI screenshots.
4. Rewrite the data, RL, metrics, database, testing, security and maintenance sections from Section 6; insert the reward equation and training setup (Appendix A.4–A.5) now.
5. Fix and verify references; add method references.
6. After the study intersections are confirmed: rebuild the network, record fixed-time baselines, retrain QL/DQL/PPO, run the matched-seed evaluation, then write the results chapter.
7. After field work: update objectives 1, 2 and 5 evidence and the calibration/validation results.

9. Final checklist

☐ No current-system SUMO, TraCI or "PyTorch DQN only" claims remain; SUMO appears only as literature.
☐ No emergency-vehicle feature, input, output, use case, field or definition remains.
☐ The study area names the two intersections (or the final confirmed ones).
☐ Title, scope and diagrams agree: RL on one selected junction; routing is a separate algorithm.
☐ The general objective is one correct sentence; all six objectives show their evidence status.
☐ Figures 1–6 have correct captions and source credits.
☐ Figures 7–16 are redrawn for the current system; Figures 17–21 replaced.
☐ The p45 FAA/ASDE-3 text and p40 garbled terms are gone.
☐ ERD and data dictionary match the real 18-table schema; RBAC described consistently.
☐ The engine model and its limits are described; synthetic checks are not called calibration.
☐ The RL observation, actions and safety rules are described; the `reward_v3` equation and its terms are written out.
☐ The training setup states the budget, warmup, episode length, training seeds and held-out evaluation seeds and scenario.
☐ The evaluation protocol states the baseline source, pedestrian rule, seeds, metrics, J1-only results, fault checks and engine version (`native-5`).
☐ Every metric uses the catalogue definition and unit; pending/unfinished demand is reported next to averages; "unavailable" is never written as zero.
☐ Leftover emergency/efficiency columns are not presented as results; JSON and timestamps are described as TEXT.
☐ Security text uses present tense for what exists and never calls hashing "encryption".
☐ Test results are dated and kept separate from research validation.
☐ References are de-duplicated, verified and all cited.
☐ Typos fixed; one figure style; list of figures and tables updated.
☐ The revised manuscript is saved as a new file; the archived original and revised PDFs stay unchanged.

Appendix A. Suggested replacement text

A.1 Scope — technology paragraph (p7)

> SMARTFLOW is implemented as a browser-based React and TypeScript application connected to a Python FastAPI backend. The backend runs SMARTFLOW's own microscopic traffic engine, the reinforcement-learning workflows and the persistent records, while Three.js and a 2D map display the simulation. The study area is a bounded road network derived from OpenStreetMap containing two signalized intersections on Osmeña Street, Tagum City, whose lanes, turns, crossings and signal timing are verified through field observation. Vehicles (cars, motorcycles, tricycles, buses and trucks) and pedestrians are simulated traffic entities generated from observed or explicitly assumed demand. Reinforcement learning controls the signal of one selected junction; routing uses a separate graph algorithm. The system does not receive live CCTV, GPS or sensor data and does not control physical traffic signals.

A.2 Architecture caption (Figure 11)

> Figure 11. SMARTFLOW architecture: browser interaction and rendering (React, Three.js), FastAPI orchestration and authentication, native traffic simulation and offline reinforcement learning (Q-learning, DQL, PPO), SQLite records, and model, recording and report files. Field observations enter through the offline Field Observer tool.

A.3 Replacement for the p45 airport passage

Signal Control Processing reads the current simulation state and applies the configured controller. Under fixed-time control, each junction follows its saved signal plan. Under reinforcement-learning control, a compatible trained policy observes the selected junction and requests one of five services (north, east, south, west or pedestrians). The engine applies the request only within its safety rules — minimum green, yellow and all-red clearance, maximum green and maximum red — and records each request, its result and any safety override for later analysis.

A.4 Reward equation (p56–57)

At each decision step *t*, of length Δt = 5 s, the reward is computed only from the controlled junction J1 over that interval:

$$r_t = 0.5 \cdot \min(1, D_t / (\Delta t / 2.0)) - 1.0 \cdot \min(1, S_t / (4 \cdot 30 \cdot \Delta t)) - 0.5 \cdot \min(1, Q_t / 30) - 0.25 \cdot \min(1, P_t / 20) - 0.05 \cdot C_t$$

where D_t is the number of vehicles that passed through J1 during the interval, normalised by the most one green lane can discharge (one vehicle per about 2.0 s); S_t is the stopped vehicle-seconds accumulated at J1's four approaches during the interval, so $S_t / (4 \cdot \Delta t)$ is the mean number of stopped vehicles per approach, normalised by a 30-vehicle queue; Q_t is the longest approach queue at the end of the interval; P_t is the number of pedestrians waiting at J1; and C_t is 1 if the signal changed phase during the interval and 0 otherwise. All terms are bounded between 0 and 1 before weighting. The reward therefore favours discharging vehicles while keeping delay, the longest queue and pedestrian waiting low, with a small cost for switching. Training uses this local reward, while the reported results use network-wide measures and J1 delay, so the policy is not judged by the quantity it was trained on alone.

A.5 Training and evaluation setup (methodology)

Each controller is trained offline for 560 episodes. Every episode starts the simulation from an empty network, runs 300 s of warmup under the fixed-time plan so that traffic reaches a developed state, and then gives the agent control for 900 s with a decision every 5 s, about 101,000 decisions in total. Training uses seeds 11, 22, 33, 44 and 55. Trained models are evaluated on a different saved scenario with seeds 101–105 that are never used in training, and each RL controller is compared with the fixed-time plan using the same scenario, seeds, warmup and measurement window. The same pedestrian rule and safety rules apply to both controllers. Results are reported for the whole network and for J1 alone, together with unfinished demand and the engine's fault checks; all runs use engine version native-5.

A.6 Database introduction (Figure 16)

SMARTFLOW uses SQLite for user, permission, scenario, run, model-registry, training-job and calibration records. The schema has 18 tables with 15 declared foreign keys. Native scenario settings are stored as one validated JSON document per scenario, while road networks, trained models, recordings, imported observation files and evaluation summaries are kept as files and referenced by the records. Live traffic state is held in the simulation engine's memory; the database stores each run's record and summary metrics. Calibration records keep a frozen copy of the baseline evidence, so later changes to a scenario or run do not alter an existing assessment.

A.7 Testing paragraph (p62–64)

Software verification uses automated tests run on 26 October 2026: 178 backend tests (407 sub-cases) covering the traffic engine rules, junction capacity and deadlock checks, signal safety rules, reward and training settings, the API, permissions and backup/restore; and 105 frontend tests with type and lint checks. These tests establish that the software behaves as specified. They do not establish that the model reproduces Tagum traffic, which requires calibration and independent validation against field observations, nor that the RL controllers outperform fixed-time control, which requires the final comparative evaluation.